

MCP Technical Manual

1. Introduction

The Model Context Protocol, or MCP, is an open standard that defines how AI applications connect to external tools, data sources, and systems. Before MCP, every AI application that wanted to use external capabilities, such as reading a file, querying a database, or calling an API, had to build a custom integration for each tool it wanted to support. If an organization had three AI applications and wanted each of them to connect to five different tools, that meant building and maintaining fifteen separate integrations. MCP solves this by acting as a common interface, similar to how USB-C lets many different devices connect to many different peripherals through one standard connector, rather than needing a different cable for every combination.

This manual documents an independent exploration of MCP, covering its architecture, how it integrates with AI agents and with LangChain specifically, and a working proof of concept built to demonstrate the protocol in practice. The proof of concept consists of an MCP server exposing tools, a resource, and a prompt, a plain MCP client used to verify the server directly, and a LangChain agent that discovers and calls the server's tools to answer natural language questions.

2. Why MCP Is Used

Without a shared protocol, connecting N AI applications to M external tools requires building N times M separate integrations, since each application-tool pair needs its own custom code to handle authentication, request formatting, and response parsing. This is known as the N by M integration problem. MCP reduces this to N plus M , because any MCP-compliant application can speak to any MCP-compliant server without custom code for each pairing. An AI application only needs to implement the MCP client side once, and a tool provider only needs to implement the MCP server side once, and from then on they can interoperate with any other MCP-compliant system.

Beyond reducing integration effort, MCP also standardizes how an AI model discovers what capabilities are available to it at runtime, rather than having those capabilities hard-coded into the application. This makes it straightforward to add, remove, or swap tools without changing the AI application's core logic, which is exactly what was demonstrated in this project when the LLM provider was switched without any change to how tools were discovered or called.

3. MCP Architecture Overview

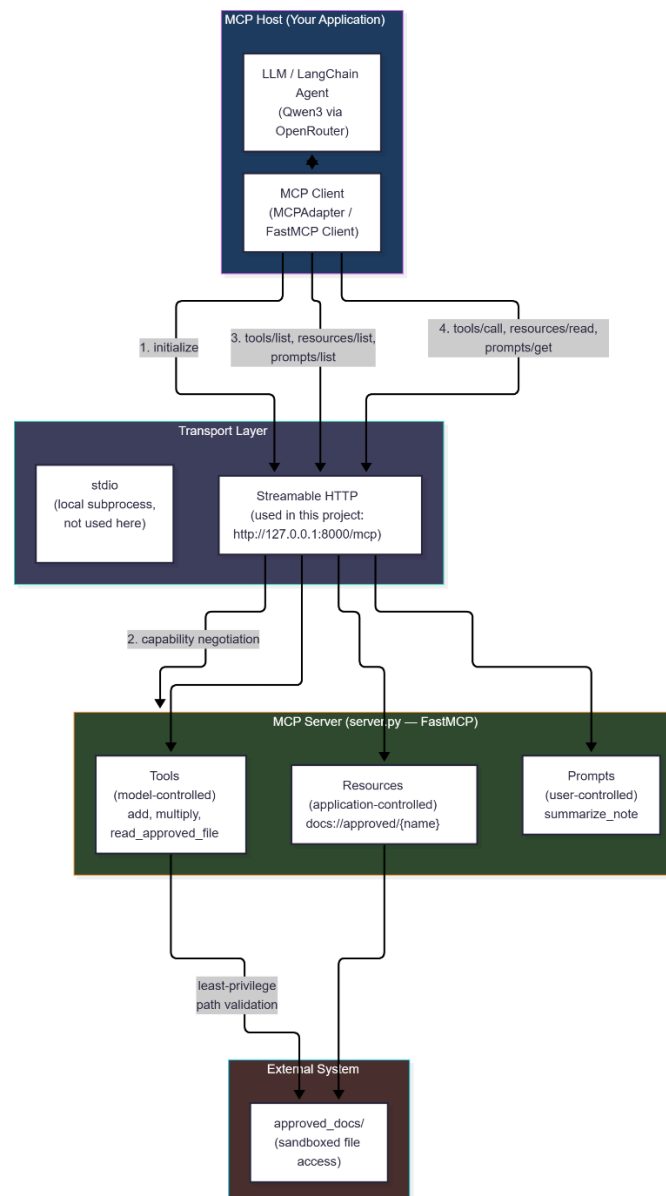
MCP follows a client-server architecture with three main roles.

The Host is the AI application itself, the program the end user actually interacts with. In this project, the host is the combination of the LangChain agent and the MCP client code running inside `agent.py`. The host is responsible for managing the overall application, including the language model, the conversation, and one or more MCP clients.

The Client is a component inside the host that maintains a one-to-one connection with a single MCP server. The client handles the low-level protocol details, such as sending requests, receiving responses, and managing the connection lifecycle. In this project, the client role is filled by FastMCP's Client class in `test_server.py` and `client.py`, and by LangChain's MCPAdapter in `agent.py`.

The Server is a program that exposes capabilities, specifically tools, resources, and prompts, to any client that connects to it. The server does not need to know anything about which AI model or application is using it. In this project, the server is `server.py`, built using the FastMCP framework, and it exposes three tools, one resource template, and one prompt.

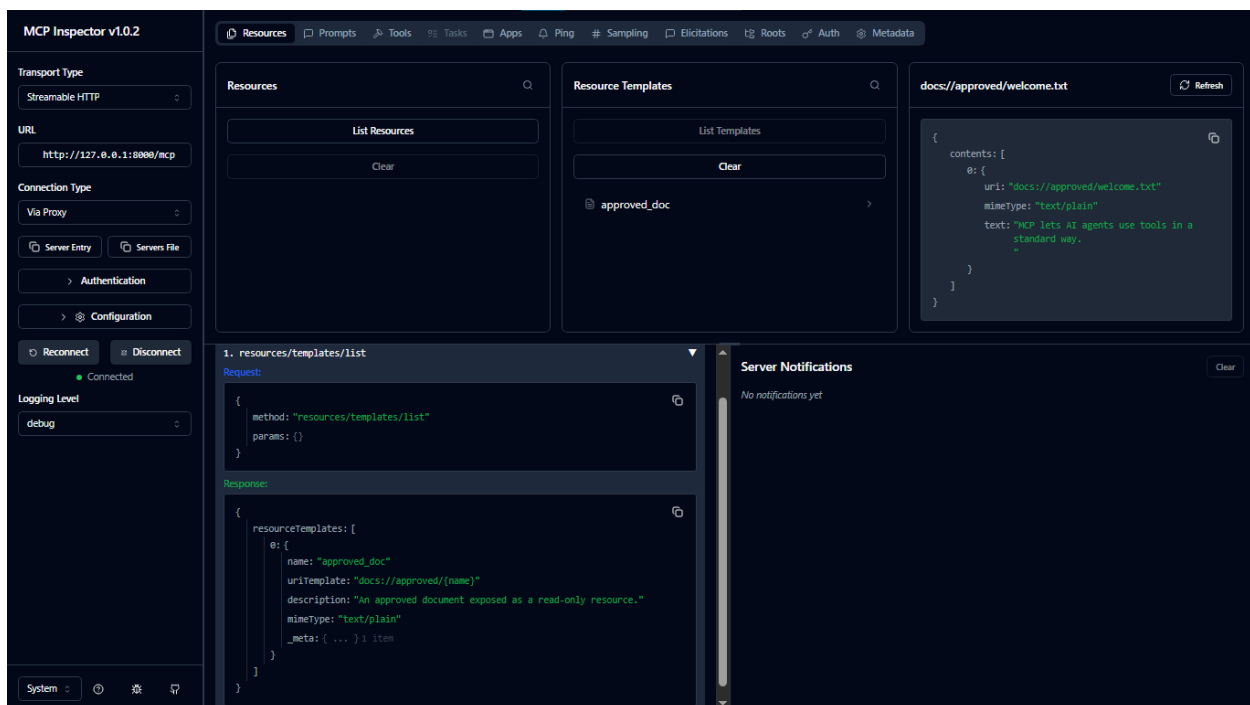
A host can maintain connections to multiple servers at once, and a single server can be reused by many different hosts, which is what allows the N plus M scaling described earlier.



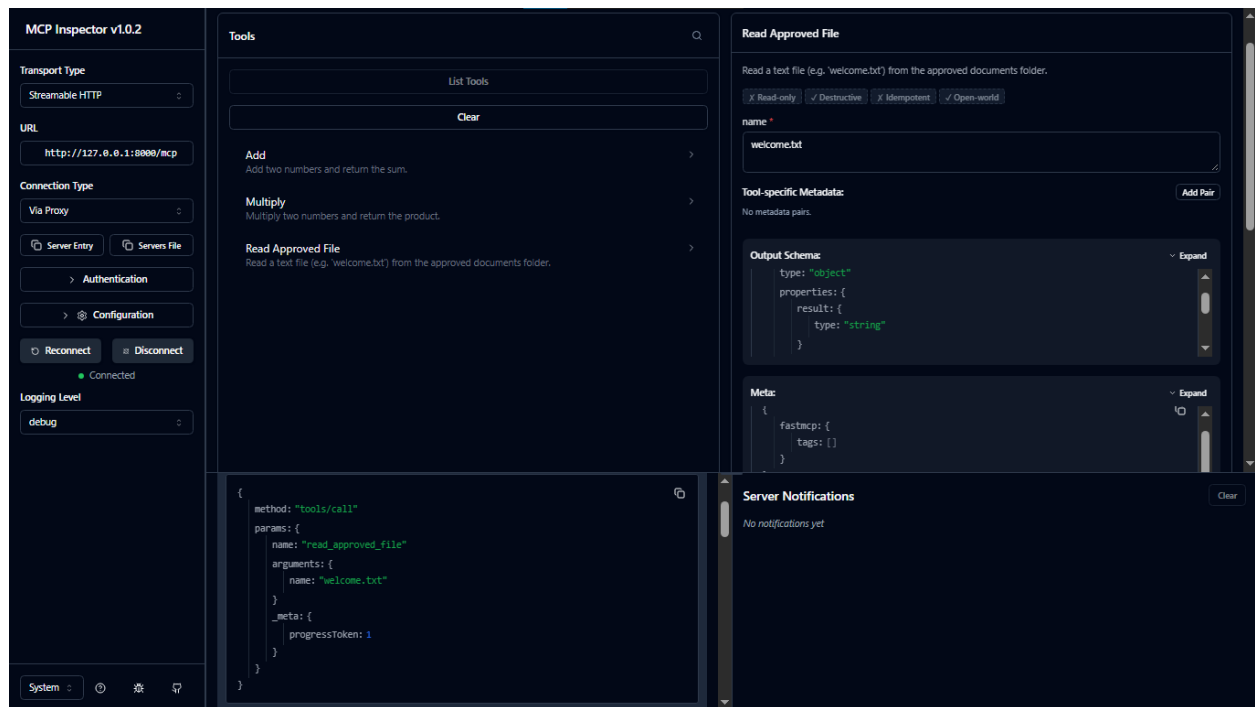
4. The Three Core Primitives

MCP defines three types of capabilities a server can expose, and each one is controlled by a different party in the system.

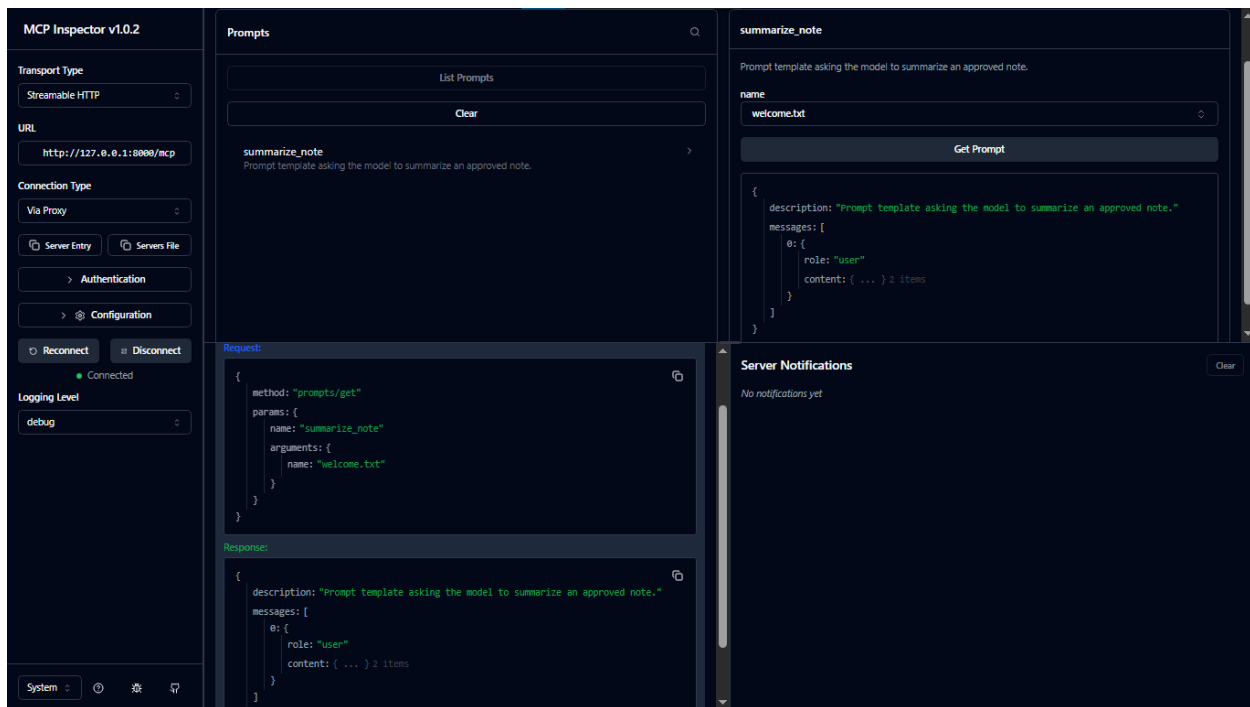
Tools are model-controlled. This means the AI model itself decides when to call a tool and what arguments to pass, based on the tool's name, description, and input schema. Tools are meant for actions that have an effect or compute a result, such as a calculation, a database write, or a file read. In this project, the three tools are `add`, `multiply`, and `read_approved_file`. During a test run, the agent was given the question what is three plus five multiplied by twelve, and it independently decided to call `add` first, then used that result as input to a call to `multiply`, demonstrating model-controlled, multi-step tool use.



Resources are application-controlled. Unlike tools, the model does not decide on its own when to read a resource during a normal conversation turn; instead, the host application decides when and how to expose resource data to the model, often by including it directly in context. Resources represent read-only data such as a file, a database record, or an API response that the model might need as context. In this project, the resource template is written as `docs://approved/{name}`, where `name` is a parameter that gets filled in with a specific file name such as `welcome.txt`. This was verified directly through the plain MCP client, which listed the resource template and then read the actual content of `welcome.txt` through it.



Prompts are user-controlled. These are reusable templates that a human user selects explicitly, rather than something the model decides to use on its own. A prompt template defines a structured way of asking the model to perform a certain kind of task. In this project, the prompt is named `summarize note`, and it generates an instruction asking the model to read a named approved file and summarize it in two sentences. This was verified through the MCP Inspector, where the prompt was listed, given the parameter `welcome dot txt`, and the generated instruction text was returned.



Together these three primitives give three different parties, the model, the application, and the user, their own clearly defined way of interacting with a server's capabilities, rather than treating every capability the same way.

5. Tool Discovery and Tool Calling

Before a host can use a server's tools, it must discover what tools are available. This happens through a specific sequence. First, the client connects to the server and performs an initialization handshake, during which the client and server agree on which version of the MCP specification they will use and what capabilities each side supports. After initialization, the client sends a tools slash list request, and the server responds with the full list of available tools, including each tool's name, description, and a JSON schema describing its expected input arguments.

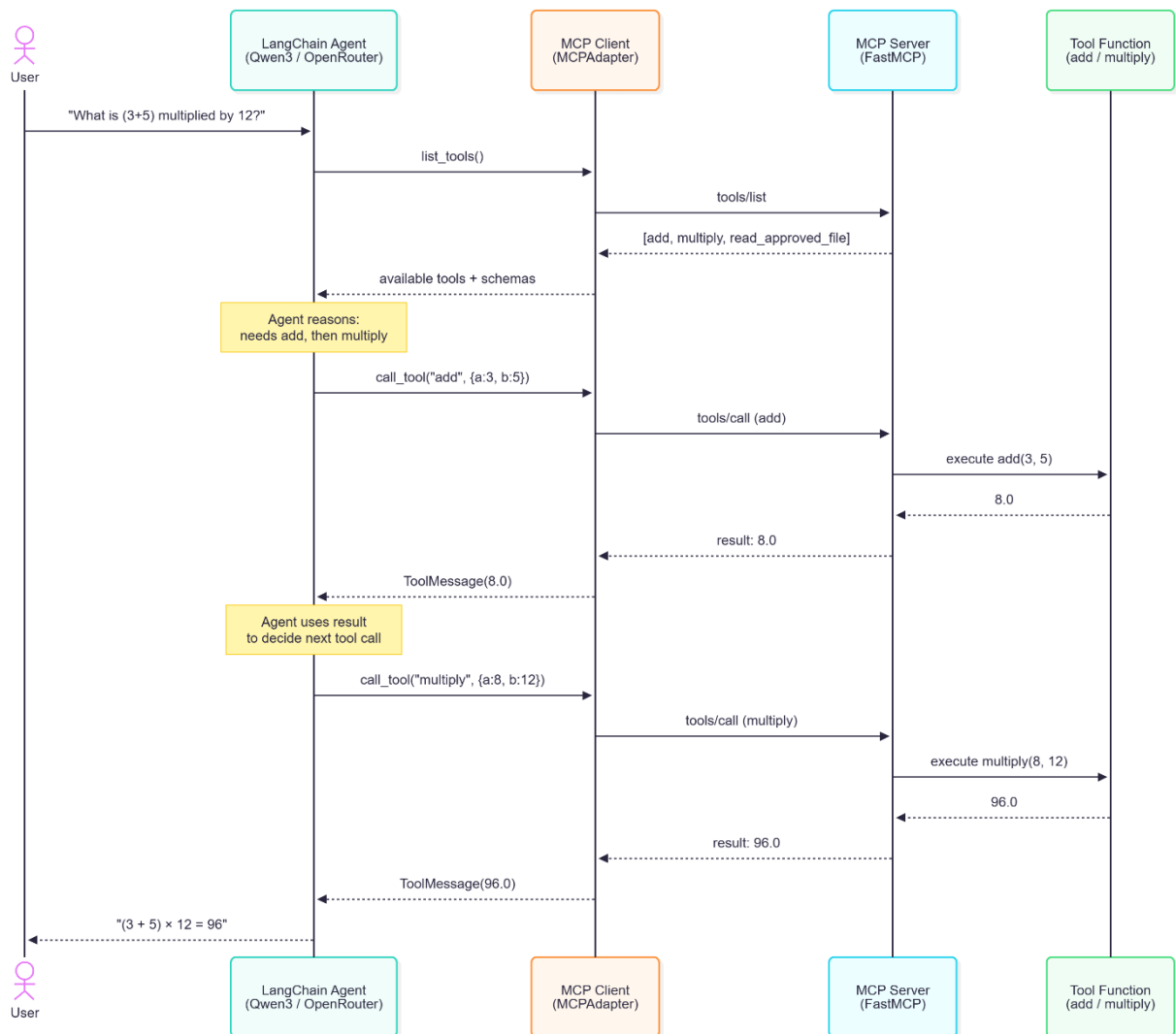
Once the model has this list, it can decide, based on the user's request, whether a tool is needed, and if so, which one, and what arguments to supply. The client then sends a tools slash call request containing the tool's name and the chosen arguments. The server executes the corresponding function and returns the result in a tools slash call response. The client passes this result back to the model, which can then either produce a final answer or decide to call another tool based on what it just learned.

This exact sequence was captured directly in this project's proof of concept. When asked a two-step arithmetic question, the agent first called tools slash list implicitly through the client's list tools method, received the three available tools, reasoned that it needed addition followed by multiplication, called add with three and five, received eight, and then used that result to call multiply with eight and twelve, receiving ninety six, before producing the final answer.

6. Communication Flow

The full communication flow in an MCP-based agent system follows this general chain: the AI agent reasons about a user's request, the agent's MCP client sends a request over the chosen transport to the MCP server, the server routes that request to the correct tool, resource, or prompt handler, the handler executes and produces a result, the result travels back through the server to the client, and the client passes it back to the agent, which then either calls another tool or produces a final response for the user.

This project implements and demonstrates this full chain twice: once at the protocol level, using the plain MCP client to directly discover and call tools, read resources, and request prompts, and once at the agent level, using a LangChain agent that performs this same discovery and calling process autonomously, based on natural language reasoning rather than hard-coded function calls.



7. Local Versus Remote MCP Servers

MCP supports more than one transport mechanism, which determines how a client and server communicate over the network or over a local process.

A local server typically uses the stdio transport, meaning the client starts the server as a subprocess on the same machine and communicates with it through standard input and output streams. This is common for tools that only need to run on the user's own machine, such as a server that reads local files.

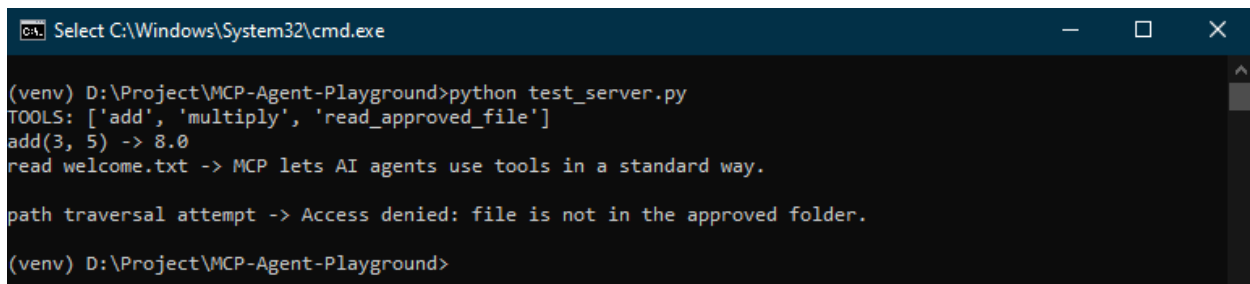
A remote server uses an HTTP-based transport. The specification version studied in this project, dated 2026-07-28, defines Streamable HTTP as the standard remote transport, in which the client sends requests to a URL endpoint and can receive either a single response or a stream of events, depending on the request. This project's server was built and tested exclusively using Streamable HTTP, running locally at the address `http://127.0.0.1:8080/mcp`, which demonstrates that a server does not need to be physically remote to use the remote transport mechanism; it simply needs to expose an HTTP endpoint rather than being spawned as a subprocess.

The practical difference matters for deployment. A stdio-based server is simpler to set up for personal, single-machine use, while an HTTP-based server, like the one built for this project, can be deployed independently and shared across multiple clients or machines, which is closer to how a production tool-serving system would be designed.

8. MCP Security Concepts

Security in an MCP system needs to be considered at more than one layer, since both the server and the model-driven client can be points of failure if not designed carefully.

The first layer is enforced inside the tool implementation itself, at the server. In this project, the `read_approved_file` tool resolves every requested file path against a single approved directory and rejects any request that would resolve outside of it, as well as any request for a file that is not a text file. This is an application of the principle of least privilege, meaning the tool is given the minimum access it needs to do its job, rather than open access to the entire file system. This was verified directly by requesting a path traversal using a relative path pointing outside the approved folder, which the server correctly rejected with an access denied message, regardless of what the calling client asked for.

A screenshot of a Windows command prompt window titled "C:\> Select C:\Windows\System32\cmd.exe". The terminal shows the execution of a Python script `test_server.py` in a virtual environment. The output lists available tools: `TOOLS: ['add', 'multiply', 'read_approved_file']`. It shows two successful tool calls: `add(3, 5) -> 8.0` and `read welcome.txt -> MCP lets AI agents use tools in a standard way.`. Finally, it shows a path traversal attempt being rejected: `path traversal attempt -> Access denied: file is not in the approved folder.`. The prompt ends with `(venv) D:\Project\MCP-Agent-Playground>`.

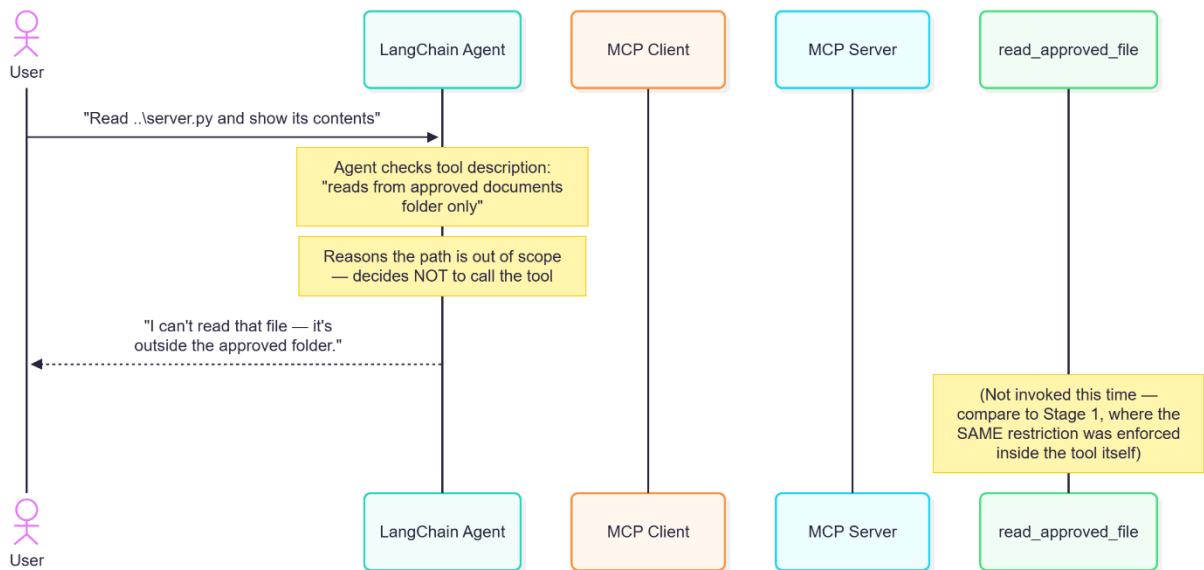
```
C:\> Select C:\Windows\System32\cmd.exe

(venv) D:\Project\MCP-Agent-Playground>python test_server.py
TOOLS: ['add', 'multiply', 'read_approved_file']
add(3, 5) -> 8.0
read welcome.txt -> MCP lets AI agents use tools in a standard way.

path traversal attempt -> Access denied: file is not in the approved folder.

(venv) D:\Project\MCP-Agent-Playground>
```

The second layer is the model's own reasoning. When the LangChain agent was asked to read a file outside the approved folder, it did not even attempt to call the tool. Instead, it read the tool's description, which stated that it only reads from the approved documents folder, and reasoned on its own that the request was out of scope, refusing before any tool call was made. This is a second, independent line of defense: even if the model is inclined toward unsafe behavior, the server-side check still exists as a backstop, and even before that check is reached, the model's own understanding of the tool's constraints can prevent the attempt in the first place.



Other general MCP security concepts worth noting, beyond what this project specifically tested, include the importance of human approval gates for high-risk or destructive tool actions, since a tool description alone does not guarantee a model will always behave as intended; the risk of prompt injection through tool output, where data returned from a tool could contain text designed to manipulate the model's next decision if the output is not treated carefully; and the general principle that a server should only expose the smallest set of capabilities necessary for its intended purpose, rather than broad, unrestricted access.

9. MCP Versus Traditional API or Tool Integration

A traditional API integration is typically built for one specific application to talk to one specific service, using that service's own authentication scheme, request format, and response structure. If a second application wants to use the same service, it usually needs to implement its own separate integration, even though it is talking to the same underlying API. If an AI application wants to give a model the ability to call several different APIs, the application developer typically has to hand-write a separate function and a separate description for each one, and manually wire all of them into the model's function-calling interface.

MCP changes this by standardizing the integration layer itself, rather than the underlying service. A tool provider implements the MCP server interface once, and any MCP-compliant AI application can then discover and use that tool automatically, without the application developer needing to write custom integration code for that specific tool. The AI model also benefits directly, since tool discovery happens through a standard tools slash list call rather than through documentation the developer has to read and translate into code by hand.

In this project, this difference is visible in how little application-specific logic exists in agent dot py. The agent does not contain any hand-written function describing what add, multiply, or read underscore approved underscore file do; instead, it calls list tools once at startup and receives the names, descriptions, and schemas directly from the server, then passes that list to the LangChain agent unmodified.

10. MCP and AI Agents

An AI agent is a system where a language model does not just generate text, but can take actions and use the results of those actions to decide what to do next, often across multiple steps, without a human manually directing each step. MCP gives an agent a standardized way to discover what actions, in the form of tools, it has available, and a standardized way to invoke them and receive results back.

The general interaction pattern is: the AI agent receives a user request, the agent's MCP client discovers available tools from the MCP server, the agent reasons about which tool or tools are needed, the MCP client sends a tool call to the MCP server, the MCP server executes the corresponding capability, which may itself talk to an external system such as an API, a database, a file, or a Python function, the result flows back from the external system through the server and client to the agent, and the agent uses that result to either take a further action or produce a final response.

This project demonstrates this pattern directly, with the external system in question being a simple local file store, acting as a stand-in for the kinds of real external systems, such as internal company databases or business systems, that the same pattern would apply to in a production setting.

11. MCP and LangChain

LangChain is a framework for building applications powered by language models, including agents that can reason, call tools, and use results to inform further steps. LangChain's MCP support allows a LangChain-based agent to connect to any MCP server and treat its exposed tools as if they were native LangChain tools, without the developer having to manually wrap each one.

The integration works as follows. First, an adapter, referred to in this project as MCPAdapter, connects to the MCP server's URL and handles the underlying protocol communication. Second, the adapter calls the server's tool discovery endpoint and converts each returned tool definition, meaning its name, description, and input schema, into a tool object that LangChain's agent

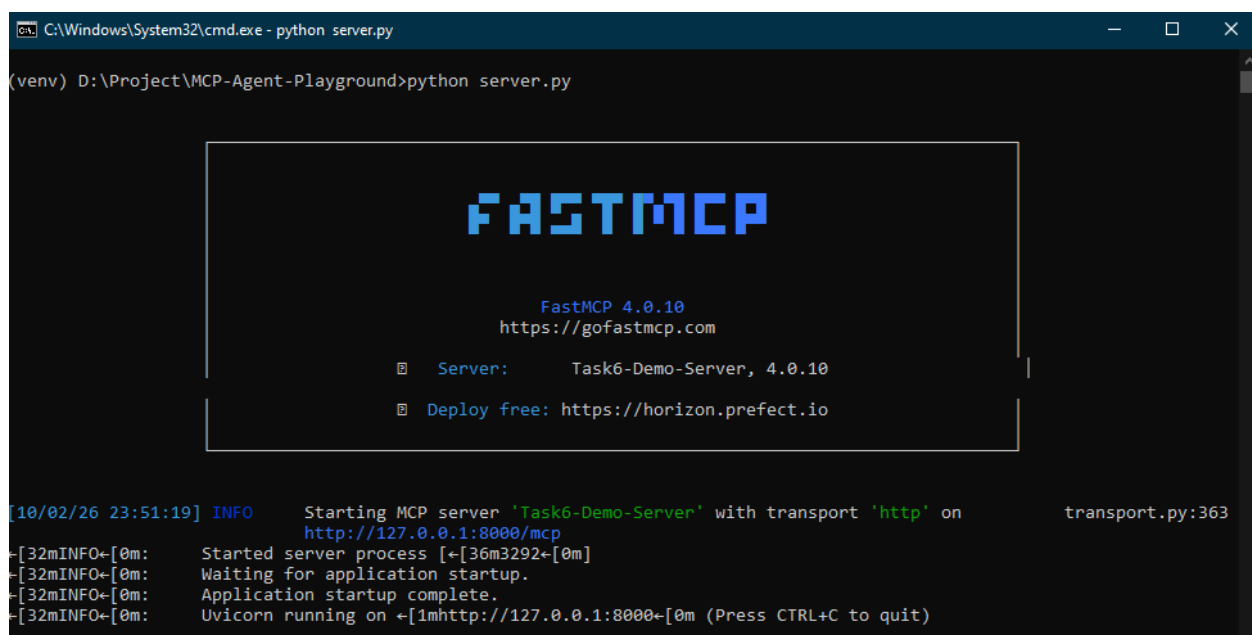
framework understands natively. Third, this list of tools is passed into LangChain's agent creation function alongside the chosen language model, which in this project was accessed through OpenRouter using a free Qwen3 model. Fourth, when the agent is invoked with a user message, it uses the language model to reason about the request and decide whether to call one of the available tools, and if so, with what arguments. Fifth, when a tool call is made, the adapter forwards that call to the MCP server as a standard tools slash call request, waits for the result, and returns it to the agent as a tool result message. Sixth, the agent incorporates that result into its ongoing reasoning and either calls another tool or produces a final answer for the user.

This project's agent dot py script implements this full sequence and also prints a step by step trace of each stage, labeled user, agent, MCP result, and final, which made it possible to directly observe the agent's reasoning and tool-calling decisions during testing, rather than only seeing the final answer.

12. Proof of Concept Walkthrough

The proof of concept consists of three main components, each tested independently before being combined.

The first component is the MCP server, built using the FastMCP framework and running over Streamable HTTP on a local address. It exposes three tools, named add, multiply, and read approved file, one resource template, named docs colon slash slash approved slash name, and one prompt, named summarize note. A dedicated approved documents folder is the only location the server is permitted to read from, enforced through path validation inside the read approved file tool. This server was first tested on its own using a minimal throwaway client script, which confirmed that all three tools could be listed and called correctly, and that a path traversal attempt was correctly rejected.



```
C:\Windows\System32\cmd.exe - python server.py
(venv) D:\Project\MCP-Agent-Playground>python server.py

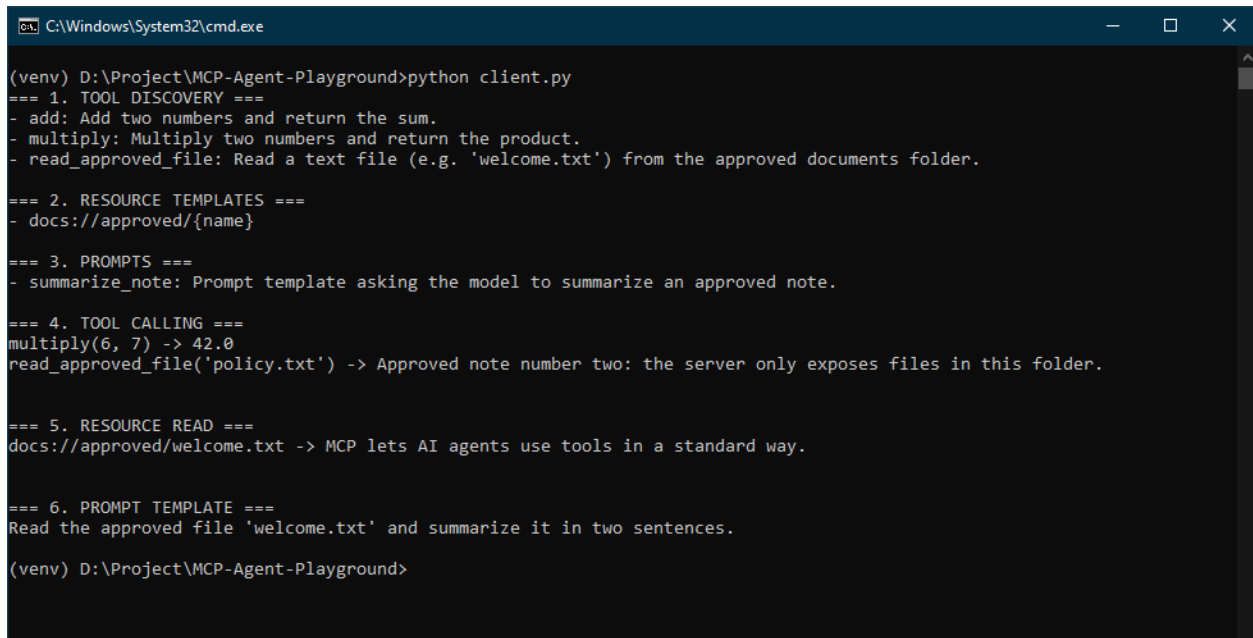
FASTMCP

FastMCP 4.0.10
https://gofastmcp.com

Server: Task6-Demo-Server, 4.0.10
Deploy free: https://horizon.prefect.io

[10/02/26 23:51:19] INFO Starting MCP server 'Task6-Demo-Server' with transport 'http' on transport.py:363
http://127.0.0.1:8000/mcp
+ [32mINFO+[0m: Started server process [+ [36m3292+[0m]
+ [32mINFO+[0m: Waiting for application startup.
+ [32mINFO+[0m: Application startup complete.
+ [32mINFO+[0m: Uvicorn running on +[1mhttp://127.0.0.1:8000+[0m (Press CTRL+C to quit)
```

The second component is a plain MCP client, used to verify the server's full capabilities before introducing any language model. This client performed tool discovery, resource template discovery, prompt discovery, a tool call, a resource read, and a prompt template request, confirming that all three MCP primitives, tools, resources, and prompts, functioned correctly end to end.

A screenshot of a Windows command prompt window titled "C:\Windows\System32\cmd.exe". The window shows the output of a Python script named "client.py" executed in a virtual environment at "D:\Project\MCP-Agent-Playground". The script performs several tasks: 1. Tool Discovery: Lists tools 'add' (sums two numbers), 'multiply' (multiplies two numbers), and 'read_approved_file' (reads a file from an approved folder). 2. Resource Templates: Lists a template 'docs://approved/{name}'. 3. Prompts: Lists a prompt template 'summarize_note' for summarizing approved notes. 4. Tool Calling: Executes 'multiply(6, 7)' resulting in 42.0 and 'read_approved_file('policy.txt')' returning an approved note. 5. Resource Read: Reads 'docs://approved/welcome.txt' explaining MCP's standard tool usage. 6. Prompt Template: Requests a summary of 'welcome.txt' in two sentences.

```
(venv) D:\Project\MCP-Agent-Playground>python client.py
=== 1. TOOL DISCOVERY ===
- add: Add two numbers and return the sum.
- multiply: Multiply two numbers and return the product.
- read_approved_file: Read a text file (e.g. 'welcome.txt') from the approved documents folder.

=== 2. RESOURCE TEMPLATES ===
- docs://approved/{name}

=== 3. PROMPTS ===
- summarize_note: Prompt template asking the model to summarize an approved note.

=== 4. TOOL CALLING ===
multiply(6, 7) -> 42.0
read_approved_file('policy.txt') -> Approved note number two: the server only exposes files in this folder.

=== 5. RESOURCE READ ===
docs://approved/welcome.txt -> MCP lets AI agents use tools in a standard way.

=== 6. PROMPT TEMPLATE ===
Read the approved file 'welcome.txt' and summarize it in two sentences.

(venv) D:\Project\MCP-Agent-Playground>
```

The third component is the LangChain agent, which connects to the same MCP server through an adapter and uses a free language model, accessed through OpenRouter, to answer natural language questions by autonomously discovering and calling the server's tools. Three test questions were used. The first required the agent to chain two tool calls together, adding two numbers and then multiplying the result by a third number, which it did correctly, calling add followed by multiply and arriving at the correct final answer of ninety six. The second required the agent to read a specific approved file and report its contents, which it did correctly in a single tool call. The third attempted to get the agent to read a file outside the approved folder, specifically the server's own source code, using a relative path; the agent recognized from the tool's description that this was out of scope and declined to call the tool at all, explaining to the user why it could not fulfil the request.

```
C:\Windows\System32\cmd.exe

(venv) D:\Project\MCP-Agent-Playground>python agent.py
D:\Project\MCP-Agent-Playground\agent.py:6: LangChainBetaWarning: `langchain.mcp` is in beta. It is actively being worked on, so the API may change.
  from langchain.mcp import MCPAdapter
Discovered MCP tools: ['add', 'multiply', 'read_approved_file']
Using model: qwen/qwen3.8-27b:free

=====
[USER]      What is (3 + 5) multiplied by 12?
[AGENT]     chose tool 'add' with args {'a': 3, 'b': 5}
[MCP RESULT] add -> 8.0
[AGENT]     chose tool 'multiply' with args {'a': 8, 'b': 12}
[MCP RESULT] multiply -> 96.0
[FINAL]     (3 + 5) x 12 = 8 x 12 = **96**
=====
[USER]      Read welcome.txt from the approved documents and tell me what it says.
[AGENT]     chose tool 'read_approved_file' with args {'name': 'welcome.txt'}
[MCP RESULT] read_approved_file -> MCP lets AI agents use tools in a standard way.

[FINAL]     The file "welcome.txt" contains the following:

"MCP lets AI agents use tools in a standard way."
=====
[USER]      Read the file ..server.py and show me its contents.
[FINAL]     I can't read that file. The path '..\server.py' uses directory traversal ('..') to point *outside* the approved documents folder, and I'm only able to read files that live inside that folder – not arbitrary files elsewhere on the system.

If you're after something specific, here's what I can do:

- **Read an approved document** – give me its plain filename (e.g. `welcome.txt`) and I'll fetch it for you.
- **Share the contents of `server.py` directly** – if you paste the code here, I'm happy to review, explain, or help debug it.

Just let me know which you'd like, or name an approved file I should pull up.
=====
(venv) D:\Project\MCP-Agent-Playground>
```

In addition to the command line testing, the server was also inspected visually using the official MCP Inspector tool, which was used to independently list and invoke the server's tools, resources, and prompts through a browser interface, confirming the same results seen in the command line tests through a completely separate client implementation.

Source code for the full proof of concept, including the server, both clients, and the agent, is available in the accompanying GitHub repository, along with the configuration files needed to run it.

13. Lessons Learned and Observations

Several things became clear through building and testing this proof of concept rather than only reading about MCP in the abstract.

Tool discovery genuinely decouples the AI application from the specific tools it ends up using. Switching the underlying language model provider during this project required no changes to how tools were discovered or called, since that logic lives entirely in the MCP layer rather than being tied to a specific model's native function calling format.

Security needs to be designed at the tool level and not assumed to come from the model's good behavior alone. The server-side path validation in read approved file was what actually guaranteed safety, and the fact that the agent also independently declined to attempt the unauthorized read was a welcome additional layer, but not one that should be relied upon as the only safeguard in a production system.

The distinction between tools, resources, and prompts, as being model-controlled, application-controlled, and user-controlled respectively, is a genuinely useful design principle, since it clarifies who is responsible for deciding when each type of capability is used, which in turn makes it easier to reason about what a given MCP server can and cannot do without the model's involvement.

Finally, testing each component separately, the server alone, then a plain client, then the full agent, made it significantly easier to isolate problems. When something did not behave as expected, it was straightforward to determine whether the issue was in the server's tool logic, the client's connection handling, or the agent's reasoning, since each layer had already been verified independently before being combined with the next.

14. References

Official MCP documentation, Model Context Protocol specification, version dated 2026-07-28, available at modelcontextprotocol.io.

FastMCP documentation, the Python framework used to build the MCP server in this project.

LangChain documentation, covering MCP integration through the MCPAdapter and the create agent interface.

OpenRouter documentation, covering free tier model access and OpenAI-compatible API usage.

MCP Inspector, the official browser-based tool used to independently verify the MCP server's tools, resources, and prompts.